Machine Learning Lecture RAG Assistant

Project Report — Retrieval-Augmented Generation with LangChain and LangGraph

1. Objective

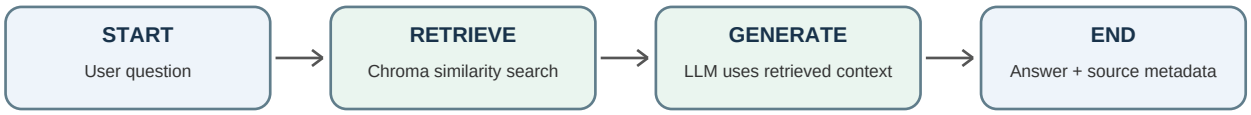
The objective is to build a question-answering application over a collection of Machine Learning lecture PDFs. The system retrieves relevant passages from the supplied documents and uses a local language model to generate answers grounded in the retrieved context. When the context is insufficient, the prompt instructs the model to respond: **“I don't know based on the provided documents.”**

2. Dataset and Implementation

The document collection contains three PDFs: ML_lecture1.pdf, ML2_Lecture2.pdf, and ML2_Lecture3.pdf. PyPDFLoader extracted 103 pages. RecursiveCharacterTextSplitter was configured with a chunk size of 1,000 characters and an overlap of 200 characters. The resulting 104 chunks were embedded using Ollama **nomic-embed-text** and stored persistently in ChromaDB under the collection **ml_lectures**.

At query time, Chroma similarity search retrieves the top 3 chunks. Ollama **llama3.2:3b** generates an answer using a prompt that restricts the response to the retrieved context. Source file names and page metadata are returned with the answer.

3. LangGraph Workflow



LangGraph state carries question, retrieved documents, and generated answer.

The graph uses a typed state containing: (1) question, (2) retrieved documents, and (3) answer. The **retrieve** node calls the Chroma retriever; the **generate** node builds the context, invokes the chat model, and returns the answer. Edges connect START → retrieve → generate → END.

4. Backend

FastAPI exposes a health/root check and a question-answering endpoint. The confirmed routes are **GET /**, **GET /health**, and **POST /ask**. Pydantic validates the request question (3–1,000 characters) and the response contains an answer and source entries (source path and page number). The interactive API documentation is available at **/docs** when the server is running.

5. Evaluation

Test	Observed result	Assessment
PDF ingestion	103 pages loaded from 3 PDFs	Passed
Chunking	104 chunks created	Passed
Vector storage	104 chunks stored in Chroma	Passed
Retrieval	Top 3 returned; specific questions were more relevant than broad questions	Functional; tune further
In-scope generation	NLP question returned an answer; sources: ML_lecture1.pdf pages 8, 7, 11	Passed initial test
Out-of-scope generation	Returned “I don't know based on the provided documents.”	Passed
LangGraph	retrieve → generate graph returned answer and sources	Passed
FastAPI	Malformed JSON produced HTTP 422; /ask also tested	Validation observed

6. Analysis and Limitations

The implementation demonstrates the complete local RAG flow: PDF ingestion, chunking, embedding, persistent vector storage, similarity retrieval, grounded generation, source metadata, and LangGraph orchestration. Initial tests showed that specific questions about lecture topics retrieved more useful passages than a broad question such as “What is Machine Learning?”. This indicates that retrieval quality depends on query specificity and should be evaluated with a larger set of representative questions. The current retriever uses top-k = 3; chunk size, overlap, and k can be tuned in future work. An NVIDIA CUDA out-of-memory issue was encountered; running Ollama with CPU execution succeeded, although CPU

inference may be slower.

The evaluation is an initial functional check rather than a comprehensive benchmark: no numeric retrieval metrics or answer-quality scores were calculated. Returned source pages should be checked against the original PDFs when assessing factual correctness.

7. How to Run

`pip install -r requirements.txt`
Install/start Ollama and pull `nomic-embed-text` and `llama3.2:3b`.
Build the vector store: `python -m app.embeddings`
Start the API: `uvicorn app.main:app --reload`
Open Swagger: `http://127.0.0.1:8000/docs`